FACULTY OF COMPUTING

UNIVERSITI TEKNOLOGI MALAYSIA

PROGRAMMING TECHNIQUE II (SECJ1023)

SEMESTER 2 2023/2024

PROJECT AND ASSIGNMENT SOFTWARE DOCUMENTATION

BUS BOOKING SYSTEM
EasyBus: SIMPLIFYING TICKET BOOKING FOR YOU

PREPARED BY:

NAME	MATRICS NUMBER
NAJMA SHAKIRAH BINTI SHAHRULZAMAN	A23CS0140
YASMIN BATRISYIA BINTI ZAHIRUDDIN	A23CS0201
NABIL AFLAH BOO BINTI MOHD YOSUF BOO YONG CHONG	A23CS0252
AINA ZAFIRAH BINTI AHMAD MUZAMIR	A23CS8014

SECTION 02

LECTURER:
DR. JOHANNA BINTI AHMAD

DATE:
30th JUNE 2024

TABLE OF CONTENTS

PART 1: INTRODUCTION

1.1 Project Synopsis

For this project, we will construct Easybus, a bus booking system that simplifies ticket booking for users. This system provides various features for passengers and administrators. Passengers can book their chosen bus, and the administrator can easily manage the bus system. In this system, we used the C++ programming language to implement a linear data structure known as an array. For example, we define an array of ten as the maximum number of bookings that each passenger can make for their ride. Our system has two distinct features for both administrators and passengers. An administrator can administer the bus system by adding new buses, editing existing buses, removing buses, and viewing the list of buses. Meanwhile, the passenger can book a ticket, cancel it, review the current bus, and rate their previous booking.

1.2 Project Objective

Project objective is a list of goals of what we want to achieve by the end of this project.

1. Develop an interactive system for passengers in booking their bus.
2. Develop a system where the admin also could make an adjustment in it.
3. Combining the passengers and admin into a single system to make the system more organized.
4. Develop a system that is friendly to users.

PART 2: SYSTEM ANALYSIS AND DESIGN

2.1 System Requirements

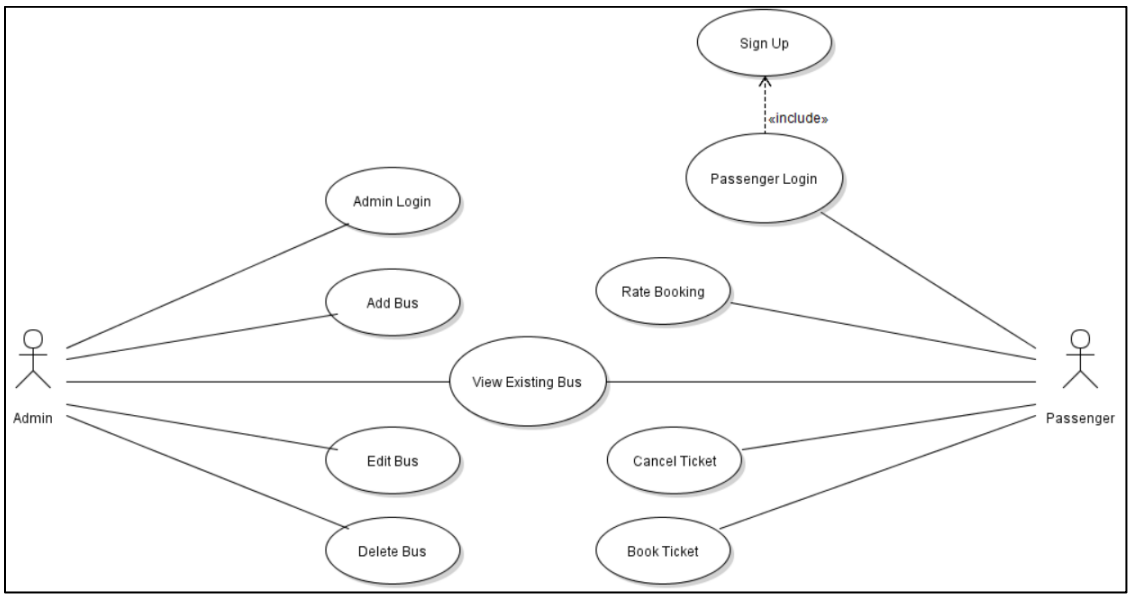


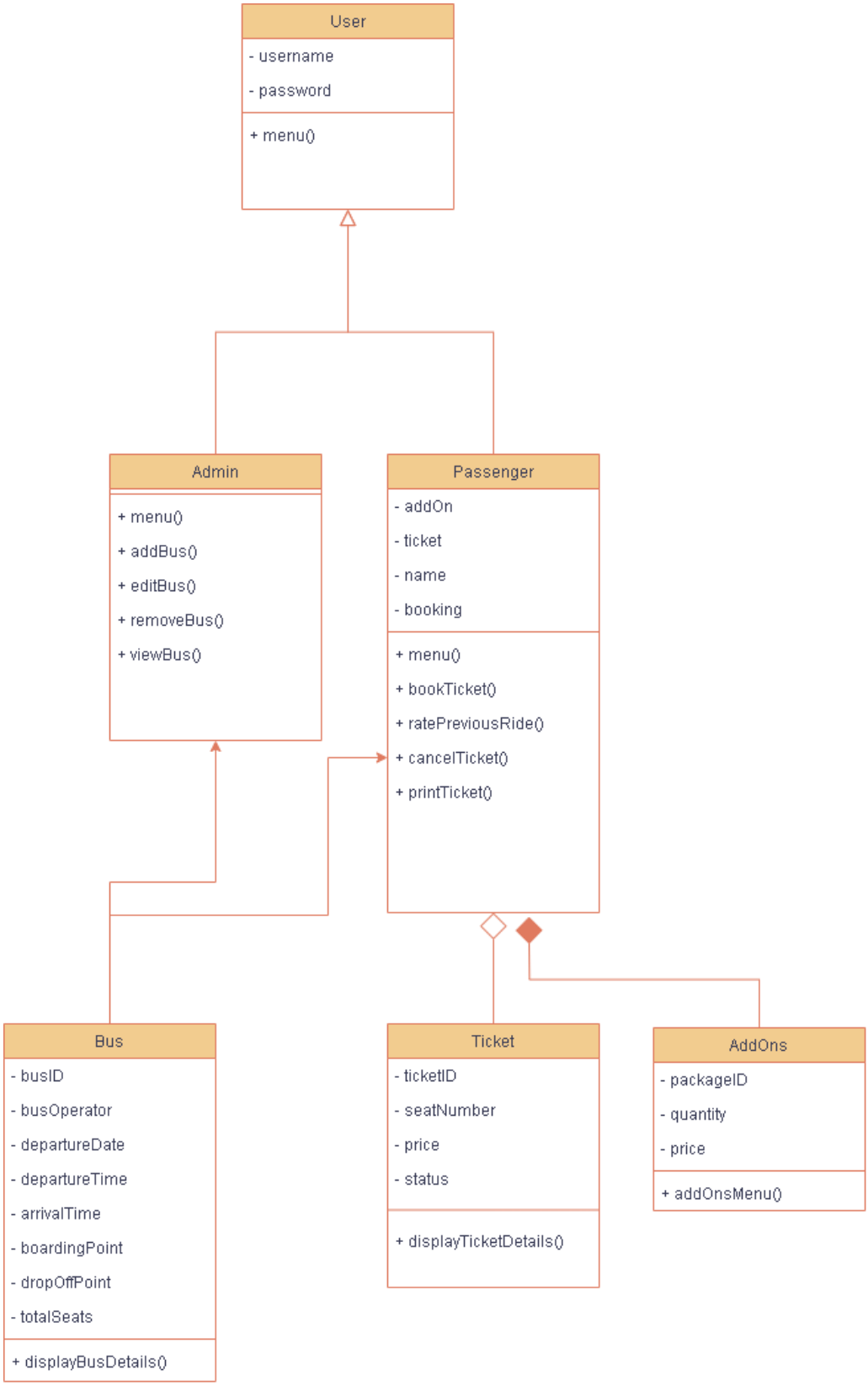
Figure 1: Use Case Diagram for Bus Ticket Booking System

Module	Use Case	Description
Admin Module	Admin Login	Allows the admin to log into the EasyBus system using the fixed credentials. This allows access to administrative functionalities such adding new buses, editing buses and deleting buses.
	Add Bus	Allows admin to add new buses into the system. This involves entering details such as bus ID, operator, departure and arrival time and date, boarding and drop off point, total seats and ticket price.
	View Existing Bus	Both admin and passengers can view the details of existing buses. This use case allows users to check for the available buses, their schedules and routes.
	Edit Bus	Allows the admin to edit the details of an existing. This includes changes in date, time, price or other bus details.
	Delete Bus	Allows admin to delete a bus from the system. This removes the bus and all

		its associated details from the database.
Passenger Module	Sign Up	A new passenger signs up for an account. This is included in the Passenger Login use case, indicating that passengers need to sign up before they can log in.
	Passenger Login	Allows passengers to login into the EasyBus system. This is required for passengers to book, view existing buses, cancel or rate bookings.
	View Existing Bus	Both admin and passengers can view the details of existing buses. This use case allows users to check for the available buses, their schedules and routes.
	Book Ticket	Allow passengers to book a ticket for a bus. This involves selecting a seat, adding options and completing the booking process.
	Cancel Ticket	Allow passengers to cancel a previously booked ticket. This use case allows the previously selected seat to be available again.
	Rate Booking	Allow the passengers to rate their booking experience.

Table 1: Description of Module and Use Case for Bus Ticket Booking System

2.2 System Design



PART 3: SYSTEM PROTOTYPE

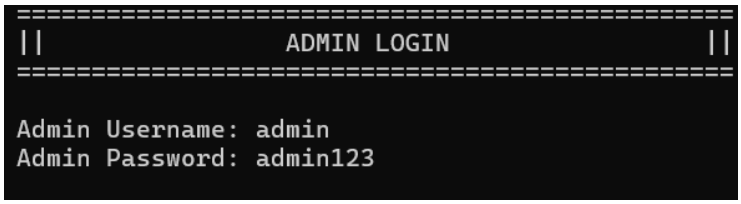
1. Main Panel: Main Menu

Screen 1: Main Menu

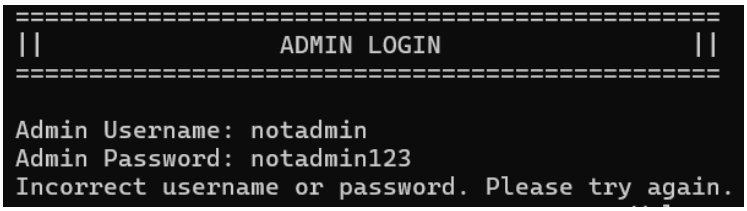
Screen 1: The main menu displays a welcome message and a prompt to ask the user if they are an admin or a passenger. If they are an admin, they would have to enter integer 1, and if they were a passenger they would have to enter integer 2. They can also enter integer 3 to exit the system. If they enter an integer other than 1-3, they will be shown an invalid input message and be prompted to reenter the option.

2. Admin Module: Admin Login

Screen 2.1: Choosing Admin from Main Menu



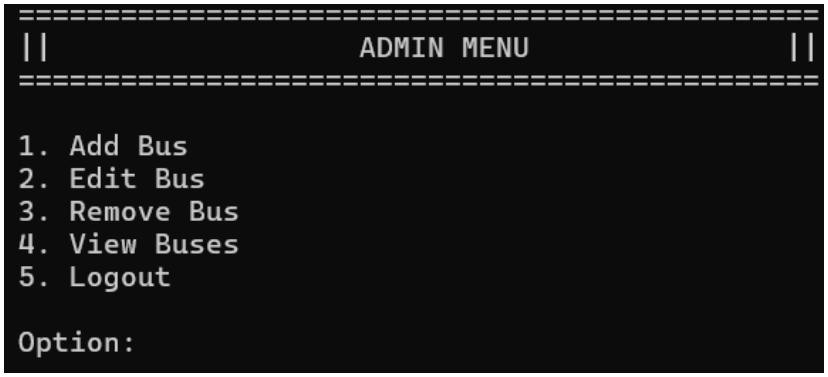
Screen 2.2: Admin Login with correct admin details



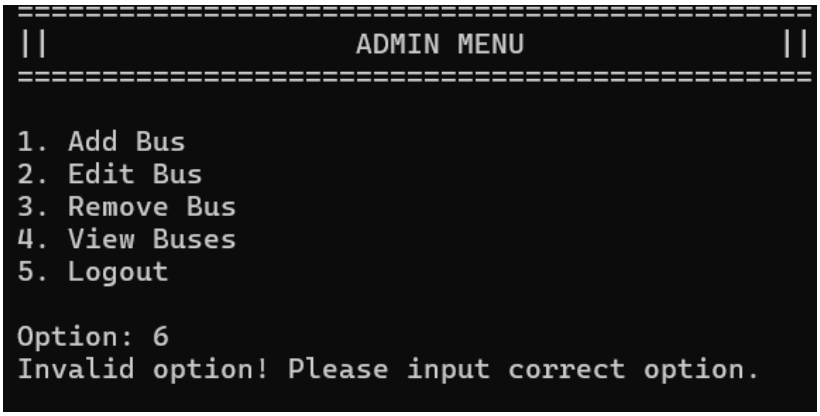
Screen 2.3: Admin Login with incorrect admin details

Screen 2: After the user chooses the first option from the main menu, they will go to the Admin Login screen. They must enter the correct login details to proceed to the admin panel. The system will compare if the input is the same as the admin details saved in the system. Otherwise, they will be given a message saying that the information they entered is incorrect and will have to reenter their details.

3. Admin Module: Admin Menu



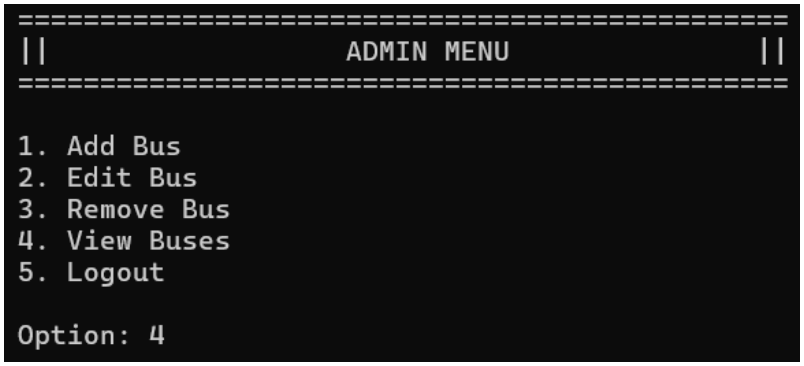
Screen 3.1: Admin Menu



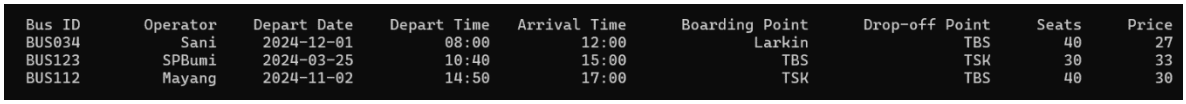
Screen 3.2: Admin Menu with incorrect input

Screen 3: The user who is the admin must insert an integer value in the range 1-5. Depending on what the admin enters, they can add a bus to the system, edit existing bus details, remove a bus from the system, view current existing buses in the system and logout to return to the main menu. If the user enters another number, the system will prompt an error message and the screen is displayed again.

4. Admin Module: View Existing bus



Screen 4.1: Admin Menu

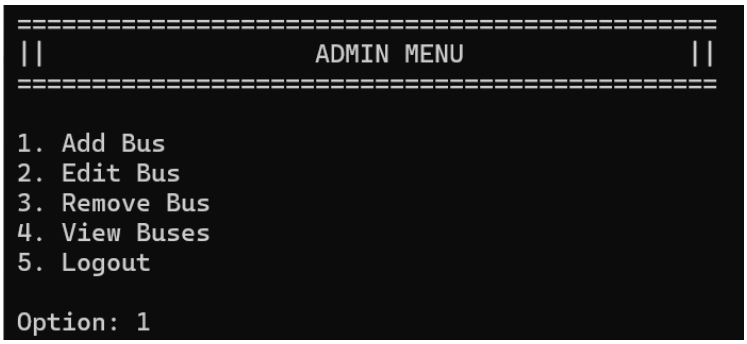


Bus ID	Operator	Depart Date	Depart Time	Arrival Time	Boarding Point	Drop-off Point	Seats	Price
BUS034	Sani	2024-12-01	08:00	12:00	Larkin	TBS	40	27
BUS123	SPBumi	2024-03-25	10:40	15:00	TBS	TSK	30	33
BUS112	Mayang	2024-11-02	14:50	17:00	TSK	TBS	40	30

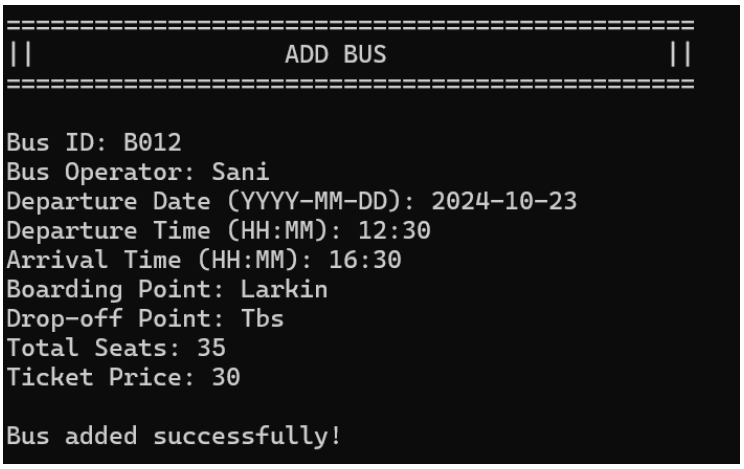
Screen 4.2: View Existing Bus

Screen 4: For this module, the admin must choose option 4 in the admin menu to view the details of the current existing bus. This allows the admin to check the available buses, their schedules and other details such as bus ID, operator, total seats and price. The system will then display the details of the current existing bus in the system.

5. Admin Module: Adding Bus



Screen 5.1: Admin Menu

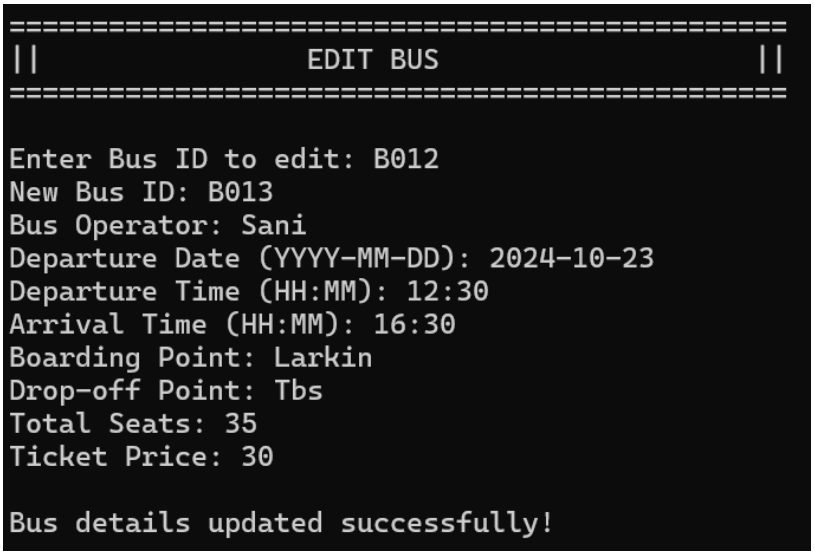


Screen 5.2: Adding bus

Screen 5: For this module, the admin must choose option number 1 on the menu page first before proceeding to the next step. After that, the admin would be brought to the add page, where the admin has to insert all of the details of the bus such as bus id, bus operator, departure date of the bus, departure and arrival time of the bus, where is the boarding and drop-off point, how many the total seats and the ticket price. Then, after completing all the details, the admin would receive a notice or message of "Bus added successfully! "Which means that the bus information has been successfully added to our system.

6. Admin Module: Edit Bus

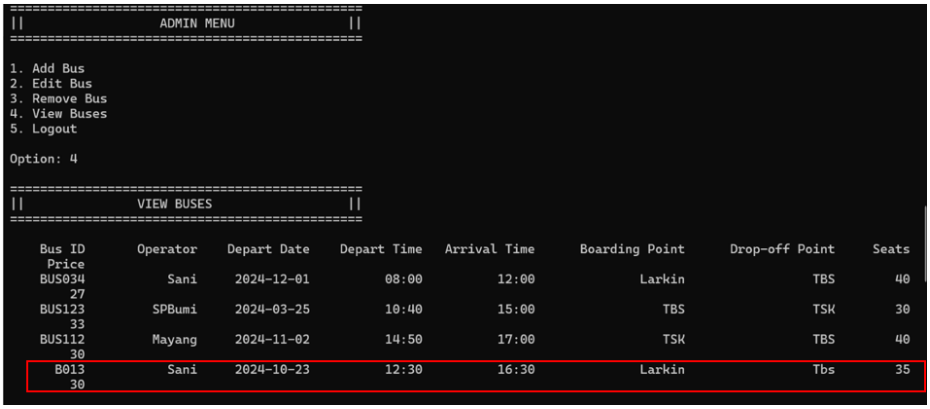
Screen 6.1: Admin Menu



Screen 6.2: Edit Bus

Screen 6: For this module, the admin must choose option number 2 on the menu page first before proceeding to the next step. After that, the admin would be brought to the edit bus page, where it could adjust which details need to be corrected. For example, referring to the screen above, the details that have been corrected are the Bus ID from B012 to B013.

7. Admin Module: Remove Bus

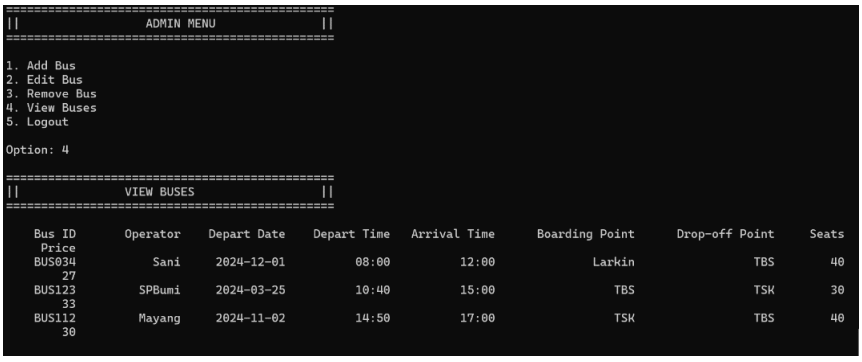


```
==========================================
||              ADMIN MENU              ||
==========================================

1. Add Bus
2. Edit Bus
3. Remove Bus
4. View Buses
5. Logout

Option: 4

==========================================
||             VIEW BUSES               ||
==========================================

    Bus ID       Operator    Depart Date    Depart Time   Arrival Time      Boarding Point      Drop-off Point      Seats
     Price
    BUS034           Sani     2024-12-01          08:00          12:00              Larkin                 TBS         40
        27
    BUS123         SPBumi     2024-03-25          10:40          15:00                 TBS                 TSK         30
        33
    BUS112         Mayang     2024-11-02          14:50          17:00                 TSK                 TBS         40
        30
     B013           Sani     2024-10-23          12:30          16:30              Larkin                 Tbs         35
        30
```

Screen 7.1: List of bus

```
==================================================
||                  ADMIN MENU                  ||
==================================================

1. Add Bus
2. Edit Bus
3. Remove Bus
4. View Buses
5. Logout

Option: 3

==================================================
||                 REMOVE BUS                   ||
==================================================

Enter Bus ID to remove: B013

Bus removed successfully!
```

Screen 7.2: Removing bus

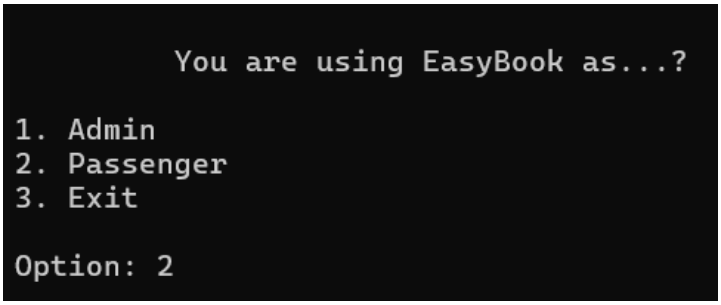


```
==========================================
||              ADMIN MENU              ||
==========================================

1. Add Bus
2. Edit Bus
3. Remove Bus
4. View Buses
5. Logout

Option: 4

==========================================
||             VIEW BUSES               ||
==========================================

    Bus ID       Operator    Depart Date    Depart Time   Arrival Time      Boarding Point      Drop-off Point      Seats
     Price
    BUS034           Sani     2024-12-01          08:00          12:00              Larkin                 TBS         40
        27
    BUS123         SPBumi     2024-03-25          10:40          15:00                 TBS                 TSK         30
        33
    BUS112         Mayang     2024-11-02          14:50          17:00                 TSK                 TBS         40
        30
```

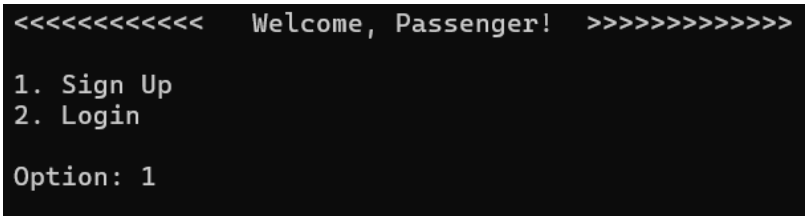
Screen 7.3: List current but after removing

Screen 7: The first screen shows the list of buses before removing. For the admin to remove bus, the admin must choose option number 3 in the menu page first before proceeding to the next page. After that, the admin would be brought up to the remove bus page, where to remove the bus from the list, the admin must insert the bus ID only. And with that, the removing bus process is done. It can be checked when the admin chooses to view the bus list again, as shown in screen 4.3 where the Bus ID B013 has been removed.

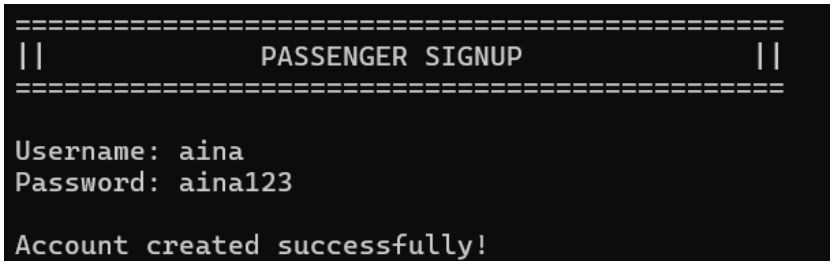
8. Passenger Module: Sign Up



Screen 8.1: Main Menu



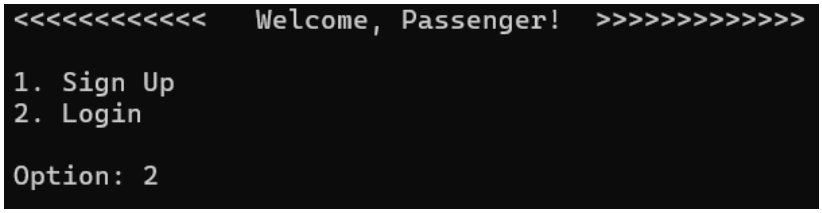
Screen 8.2: Passenger Panel



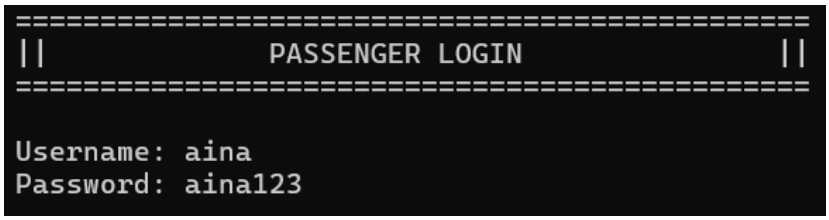
Screen 8.3: Passenger Sign Up

Screen 8: For this module, passengers will need to choose option 2 in the Main Menu to access the passenger panel of the system. Then, they will need to choose option 1 which is the Sign-Up option to create a new account. The system will then prompt the passenger to enter their credentials which are username and password. Passenger's username and password will be stored in the system's database and their account will be created successfully.

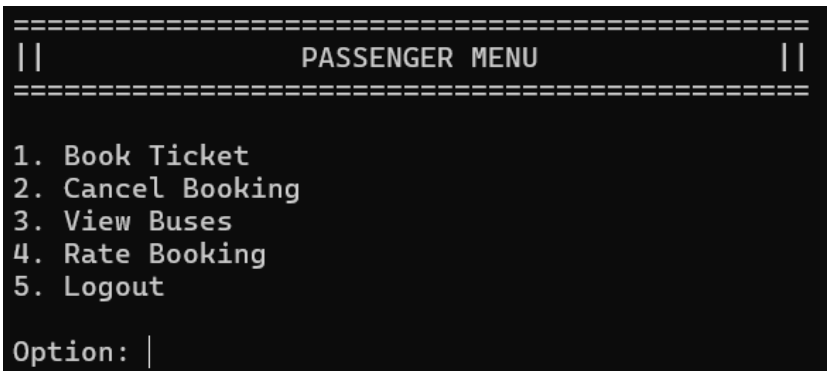
9. Passenger Module: Passenger Login



Screen 9.1: Passenger Panel



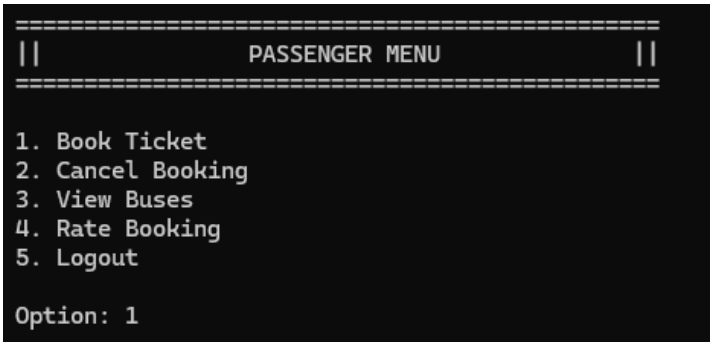
Screen 9.2: Passenger Login



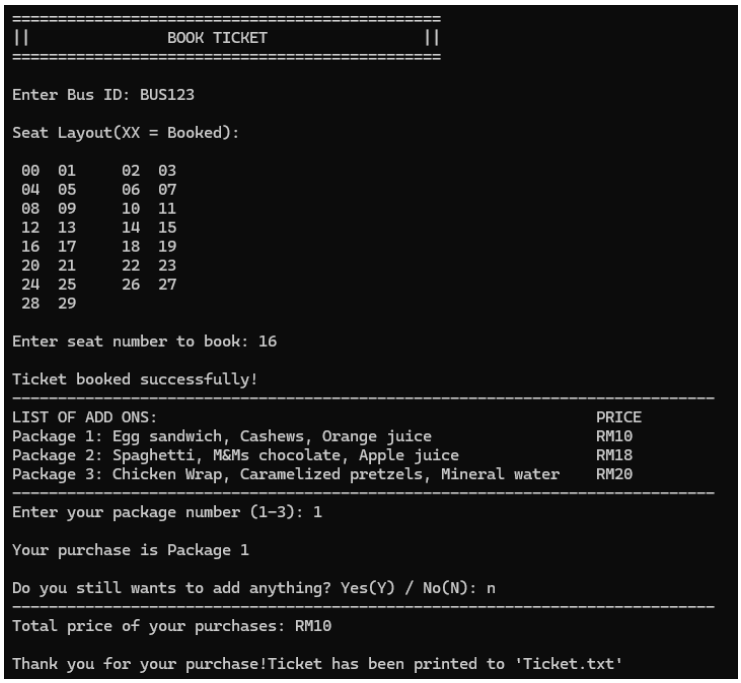
Screen 9.3: Passenger Menu

Screen 9: For this module, passengers will need to choose option 2 in the Main Menu to access the passenger panel of the system. Then, they will need to choose option 2 which is the Login option to enter the passenger Main Menu. The system will then prompt the passenger to enter their credentials which are username and password. If the username and password is correct, the system will display the passenger menu which allows the passenger to choose options like book tickets, cancel booking, view existing buses and rate booking.

10. Passenger Module: Book Ticket



Screen 10.1: Passenger Menu



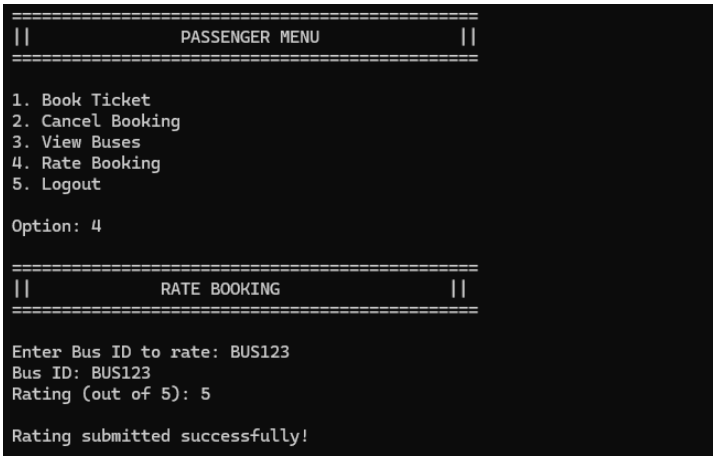
Screen 10.2: Passenger Book Ticket

```
EasyBus Booking Confirmation

Passenger Info:
Name: aflah
Payment Status: Paid

Bus Details:
Bus ID: BUS123
Seat Number: 16
Operator: SPBumi
Departure Date: 2024-03-25
Departure Time: 10:40
Arrival Time: 15:00
Boarding Point: TBS
Drop-off Point: TSK
Ticket Price: RM33
Add Ons Price: RM10



Total Price: RM43
```

Notepad 1: Ticket Receipt

To book a ticket using this module, passengers must select option 1 from the passenger menu. After that, passengers will be directed to the book ticket page, where they must enter their bus ID and selected seat number. Passengers can select whether or not they require add-ons for their ticket, and the total price for the add-ons selected will be displayed. The ticket receipt will be sent to their notepad, such as Notepad 1, where they can review their booking.

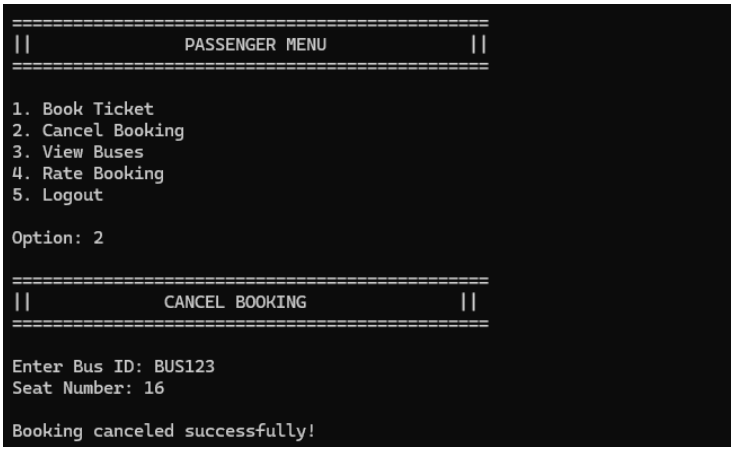
11. Passenger Module: Rate Booking



Screen 11.1: Passenger Rate Booking

Screen 11: Passengers can rate their booking by selecting Option 4 from the passenger menu page. It will then display the Rate Booking page, where they must enter the prior Bus ID and rate their booking on a scale of 1 to 5. The system then displays a notification indicating that their rating has been successfully submitted.

12. Passenger Module: Cancel Booking



Screen 12.1: Passenger Cancel Booking

Screen 12: Passengers can cancel their reservations by selecting option 2 from the passenger menu page. It will display the cancel booking page, and passengers must input the Bus ID that they wish to cancel. The system will then demand for their seat number and display a message informing them that the booking has been successfully cancelled.

PART 4: APPENDIX

copy of the source code

```
#include <iostream>
#include <string>
#include <iomanip>
#include <fstream>
const int MAX_PASSENGERS = 100;
int numBuses = 0;
int numPassengers = 0;

using namespace std;

// Ticket class declaration
class Ticket {
private:
   string busID;
   int seatNumber;
   string passengerName;
   double price;

public:
   Ticket();
   Ticket(string, int, string, double);
   ~Ticket();
   void displayTicketDetails() const;
};

// Ticket class definition
Ticket::Ticket() {}
Ticket::Ticket(string id, int seat, string name, double p) : busID(id),
seatNumber(seat), passengerName(name), price(p) {}
Ticket::~Ticket() {}
void Ticket::displayTicketDetails() const {
   cout << "Ticket Details:\nBus ID: " << busID << "\nSeat Number: " << seatNumber
<< "\nPassenger Name: " << passengerName << "\nPrice: " << price << endl;
}
//Bus class declaration
class Bus
{
private:

        string busID;
   string busOperator;
```

```
    string departureDate;
    string departureTime;
    string arrivalTime;
    string boardingPoint;
    string dropOffPoint;
    int totalSeats;
    bool seatMap[40];
    double ticketPrice;

public:
    Bus();
    Bus(string, string, string, string, string, string, string, int, double);
    ~Bus();
    string getBusID() const;
    double getTicketPrice() const;
    void displayBusDetails() const;
    int availableSeats() const;
    void displaySeatMap() const;
    bool bookSeat(int);
    void cancelSeat(int);
    void setBusID(string);
    void setBusOperator(string);
    void setDepartureDate(string);
    void setDepartureTime(string);
    void setArrivalTime(string);
    void setBoardingPoint(string);
    void setDropOffPoint(string);
    void setTotalSeats(int);
    string getBusID();
    string getBusOperator();
    string getDepartureDate();
    string getDepartureTime();
    string getArrivalTime();
    string getBoardingPoint();
    string getDropOffPoint();
    int getTotalSeats();
    void setTicketPrice(double price);
    double getTicketPrice();
};

//Bus class definition
Bus::Bus() {}
Bus::Bus(string id, string op, string depDate, string depTime, string arrTime, string
board, string drop, int seats, double price)
```

```
    : busID(id), busOperator(op), departureDate(depDate),
departureTime(depTime), arrivalTime(arrTime), boardingPoint(board),
dropOffPoint(drop), totalSeats(seats), ticketPrice(price)
  {
    for (int i = 0; i < seats; ++i)
    {
      seatMap[i] = false;
    }
  }
Bus::~Bus(){}
string Bus::getBusID() const { return busID; }
double Bus::getTicketPrice() const { return ticketPrice; }
void Bus::displayBusDetails() const
  {
    cout << setfill(' ') << setw(10) << setfill(' ') << busID << setw(15) << busOperator <<
setw(15) << departureDate << setw(15) << departureTime
       << setw(15) << arrivalTime << setw(20) << boardingPoint << setw(20) <<
dropOffPoint << setw(10)<< availableSeats() << setw(10) << ticketPrice << endl;
  }

int Bus::availableSeats() const
  {
    int count = 0;
    for (int i = 0; i < totalSeats; ++i)
    {
      if (!seatMap[i])
        count++;
    }
    return count;
  }

void Bus::displaySeatMap() const
  {
    cout << "\nSeat Layout(XX = Booked):\n\n";
    for (int i = 0; i < totalSeats; ++i)
    {
      if (i % 4 == 0 && i != 0)
        cout << "\n";
      if (i % 2 == 0 && i != 0 && i % 4 != 0)
        cout << "   ";

      if (seatMap[i])
      {
        cout << " XX";
      }
```

```
            else
            {
                cout << " " << setw(2) << setfill('0') << i;
            }
            cout << " ";
        }
        cout << "\n";
        cout << setfill(' ');
    }

bool Bus::bookSeat(int seatNumber)
    {
        if (seatNumber >= 0 && seatNumber < totalSeats && !seatMap[seatNumber])
        {
            seatMap[seatNumber] = true;
            return true;
        }
        return false;
    }

void Bus::cancelSeat(int seatNumber)
    {
        if (seatNumber >= 0 && seatNumber < totalSeats && seatMap[seatNumber])
        {
            seatMap[seatNumber] = false;
        }
    }

void Bus::setBusID(string bid) { busID = bid; }
void Bus::setBusOperator(string bo) { busOperator = bo; }
void Bus::setDepartureDate(string dd) { departureDate = dd; }
void Bus::setDepartureTime(string dt) { departureTime = dt; }
void Bus::setArrivalTime(string at) { arrivalTime = at; }
void Bus::setBoardingPoint(string bp) { boardingPoint = bp; }
void Bus::setDropOffPoint(string dp) { dropOffPoint = dp; }
void Bus::setTotalSeats(int ts) { totalSeats = ts; }
string Bus::getBusID() { return busID; }
string Bus::getBusOperator() { return busOperator; }
string Bus::getDepartureDate() { return departureDate; }
string Bus::getDepartureTime() { return departureTime; }
string Bus::getArrivalTime() { return arrivalTime; }
string Bus::getBoardingPoint() { return boardingPoint; }
string Bus::getDropOffPoint() { return dropOffPoint; }
int Bus::getTotalSeats() { return totalSeats; }
void Bus::setTicketPrice(double price) { ticketPrice = price; }
```

```
double Bus::getTicketPrice() { return ticketPrice; }

Bus busList[10];

//AddOns Class Declaration
class AddOns{
  private:
    int num;
    char choose;
    double totalPrice;

  public:
    AddOns();
    double getTotalPrice() const;
    void orderAddOns();

};

//AddOns Class Definition
AddOns::AddOns(): totalPrice (0.0) {}

double AddOns::getTotalPrice() const { return totalPrice; }

void AddOns::orderAddOns (){
  cout<<"--------------------------------------------------------------------------------"<<endl;
  cout<<"LIST OF ADD ONS:  \t\t\t\t\t\tPRICE"<<endl;

  cout<<"Package 1: Egg sandwich, Cashews, Orange juice\t\t\tRM10"<<endl;
  cout<<"Package 2: Spaghetti, M&Ms chocolate, Apple juice\t\tRM18"<<endl;
  cout<<"Package 3: Chicken Wrap, Caramelized pretzels, Mineral
water\tRM20"<<endl;

  cout<<"--------------------------------------------------------------------------------"<<endl;

  do{
  cout<<"Enter your package number (1-3): ";
  cin>>num;

  switch (num){
    case 1:
      cout<<"\nYour purchase is Package 1\n";
      totalPrice += 10;
      break;
    case 2:
      cout<<"\nYour purchase is Package 2\n";
```

```
      totalPrice += 18;
      break;
    case 3:
      cout<<"\nYour purchase is Package 3\n";
      totalPrice += 20;
      break;
    default:
            cout << "Invalid package number. Please enter a number between 1 and
3.\n";
  }

  cout<<"\nDo you still wants to add anything? Yes(Y) / No(N): ";
  cin>>choose;

  }while (choose == 'Y' || choose == 'y');

  cout << "--------------------------------------------------------------------------------" << endl;
    cout << "Total price of your purchases: RM" << totalPrice << endl;
    cout << "\n\nThank you for your purchase!";
}

//User class declaration
class User
{
protected:
    string username;
    string password;

public:
    User();
    User(string, string);
    ~User();

    virtual void menu() = 0;
          void setUsername(string);
          void setPassword(string);
    string getUsername() const;
    string getPassword() const;
};

//User class definition
User::User(){}
User::User(string user, string pass) : username(user), password(pass) {}
User::~User(){}
void User::setUsername(string un){ username = un;}
```

```
void User::setPassword(string pw){ password = pw;}
string User::getUsername() const { return username; }
string User::getPassword() const { return password; }

//Admin class declaration
class Admin : public User
{
public:
   Admin();
   Admin(string, string);
   ~Admin();

   void menu() override;

   void addBus();
   void editBus();
   void removeBus();
   void viewBuses();
};

//Admin class definition
Admin::Admin(){}
Admin::Admin(string user, string pass) : User(user, pass) {}
Admin::~Admin(){}
void Admin::menu()
{
   int choice;
   while (true)
   {
      cout << "\n================================================\n";
      cout << "||                ADMIN MENU                ||\n";
      cout << "================================================\n\n";
      cout << "1. Add Bus\n";
      cout << "2. Edit Bus\n";
      cout << "3. Remove Bus\n";
      cout << "4. View Buses\n";
      cout << "5. Logout\n";
      cout << "\nOption: ";
      cin >> choice;

      switch (choice)
      {
      case 1:
         addBus();
         break;
```

```
        case 2:
            editBus();
            break;
        case 3:
            removeBus();
            break;
        case 4:
            viewBuses();
            break;
        case 5:
            return;
        default:
            cout << "Invalid option! Please input correct option.\n";
        }
    }
}

void Admin::addBus()
{
    string busID, busOperator, departureDate, departureTime, arrivalTime,
boardingPoint, dropOffPoint;
    int totalSeats;
    double ticketPrice;

    cout << "\n=============================================\n";
    cout << "||              ADD BUS                ||\n";
    cout << "=============================================\n\n";
    cout << "Bus ID: ";
    cin >> busID;

    // Check if bus ID already exists
    for (int i = 0; i < numBuses; ++i)
    {
        if (busList[i].getBusID() == busID)
        {
            cout << "\nBus ID already exists. Please choose a different ID.\n";
            return;
        }
    }

    cout << "Bus Operator: ";
    cin >> busOperator;
    cout << "Departure Date (YYYY-MM-DD): ";
    cin >> departureDate;
    cout << "Departure Time (HH:MM): ";
```

```
    cin >> departureTime;
    cout << "Arrival Time (HH:MM): ";
    cin >> arrivalTime;
    cout << "Boarding Point: ";
    cin >> boardingPoint;
    cout << "Drop-off Point: ";
    cin >> dropOffPoint;
    cout << "Total Seats: ";
    cin >> totalSeats;
    cout << "Ticket Price: ";
    cin >> ticketPrice;

    if (numBuses < 10)
    {
        busList[numBuses++] = Bus(busID, busOperator, departureDate, departureTime,
arrivalTime, boardingPoint, dropOffPoint, totalSeats, ticketPrice);
        cout << "\nBus added successfully!\n";
    }
    else
    {
        cout << "\nCannot add more buses. Maximum limit reached.\n";
    }
}

void Admin::editBus()
{
    string busID;
    cout << "\n==============================================\n";
    cout << "||              EDIT BUS                ||\n";
    cout << "==============================================\n\n";
    cout << "Enter Bus ID to edit: ";
    cin >> busID;

    for (int i = 0; i < numBuses; ++i)
    {
        if (busList[i].getBusID() == busID)
        {
            string newBusID, busOperator, departureDate, departureTime, arrivalTime,
boardingPoint, dropOffPoint;
            int totalSeats;
            double ticketPrice;

            cout << "New Bus ID: ";
            cin >> newBusID;
```

```
        for (int j = 0; j < numBuses; ++j)
        {
            if (i != j && busList[j].getBusID() == newBusID)
            {
                cout << "\nBus ID already exists. Please choose a different ID.\n";
                return;
            }
        }

        busList[i].setBusID(newBusID);

        cout << "Bus Operator: ";
        cin >> busOperator;
        cout << "Departure Date (YYYY-MM-DD): ";
        cin >> departureDate;
        cout << "Departure Time (HH:MM): ";
        cin >> departureTime;
        cout << "Arrival Time (HH:MM): ";
        cin >> arrivalTime;
        cout << "Boarding Point: ";
        cin >> boardingPoint;
        cout << "Drop-off Point: ";
        cin >> dropOffPoint;
        cout << "Total Seats: ";
        cin >> totalSeats;
        cout << "Ticket Price: ";
        cin >> ticketPrice;

        busList[i] = Bus(newBusID, busOperator, departureDate, departureTime,
arrivalTime, boardingPoint, dropOffPoint, totalSeats, ticketPrice);
        cout << "\nBus details updated successfully!\n";
        return;
      }
   }

   cout << "\nBus ID not found. Please enter a valid Bus ID.\n";
}

void Admin::removeBus()
{
   string busID;
   cout << "\n============================================\n";
   cout << "||              REMOVE BUS                ||\n";
   cout << "============================================\n\n";
   cout << "Enter Bus ID to remove: ";
```

```
    cin >> busID;

    for (int i = 0; i < numBuses; ++i)
    {
        if (busList[i].getBusID() == busID)
        {
            for (int j = i; j < numBuses - 1; ++j)
            {
                busList[j] = busList[j + 1];
            }
            numBuses--;
            cout << "\nBus removed successfully!\n";
            return;
        }
    }

    cout << "\nBus ID not found. Please enter a valid Bus ID.\n";
}

void Admin::viewBuses()
{
    cout << "\n==============================================\n";
    cout << "||              VIEW BUSES                  ||\n";
    cout << "==============================================\n\n";
    cout << setw(10) << "Bus ID" << setw(15) << "Operator" << setw(15) << "Depart Date"
<< setw(15) << "Depart Time"
         << setw(15) << "Arrival Time" << setw(20) << "Boarding Point" << setw(20) <<
"Drop-off Point" << setw(10) << "Seats" << setw(10) << "Price" << endl;

    for (int i = 0; i < numBuses; ++i)
    {
        busList[i].displayBusDetails();
    }
}

//Passenger class declaration
class Passenger : public User
{
private:
            AddOns addOn;
            Ticket* ticket;
            string name;
    struct Booking
    {
        string busID;
```

```
        string seatNumber;
        string rating;
        string paymentStatus;
    };

    Booking bookings[10]; // Assuming max 10 bookings
    int numBookings;

public:
    Passenger();
    Passenger(string, string);
            ~Passenger();
    void menu() override;

    void bookTicket();
    void cancelBooking();
    void viewBuses();
    void rateBooking();
    void printTicket(const Booking &booking, double);
    void displayPassengerDetails() const;
};

//Passenger class definition
Passenger::Passenger() : numBookings(0) {}
Passenger::Passenger(string user, string pass) : User(user, pass), numBookings(0) {}
Passenger::~Passenger(){}
void Passenger::menu()
{
    int choice;
    while (true)
    {
        cout << "\n===============================================\n";
        cout << "||              PASSENGER MENU              ||\n";
        cout << "===============================================\n\n";
        cout << "1. Book Ticket\n";
        cout << "2. Cancel Booking\n";
        cout << "3. View Buses\n";
        cout << "4. Rate Booking\n";
        cout << "5. Logout\n";
        cout << "\nOption: ";
        cin >> choice;

        switch (choice)
        {
        case 1:
```

```
            bookTicket();
            break;
        case 2:
            cancelBooking();
            break;
        case 3:
            viewBuses();
            break;
        case 4:
            rateBooking();
            break;
        case 5:
            return;
        default:
            cout << "Invalid option! Please input correct option.\n";
        }
    }
}

void Passenger::bookTicket()
{
    string busID;
    int seatNumber;

    cout << "\n===============================================\n";
    cout << "||              BOOK TICKET                ||\n";
    cout << "===============================================\n\n";
    cout << "Enter Bus ID: ";
    cin >> busID;

    // Find the bus in the busList
    for (int i = 0; i < numBuses; ++i)
    {
        if (busList[i].getBusID() == busID)
        {
            // Display available seats
            busList[i].displaySeatMap();

            // Ask for seat number
            cout << "\nEnter seat number to book: ";
            cin >> seatNumber;

            if (seatNumber >= 0 && seatNumber < busList[i].getTotalSeats())
            {
                if (busList[i].bookSeat(seatNumber))
```

```
            {
                // Successfully booked
                cout << "\nTicket booked successfully!\n";

                // Add booking details to passenger's booking history
                if (numBookings < 10)
                {
                    bookings[numBookings].busID = busID;
                    bookings[numBookings].seatNumber = to_string(seatNumber);
                    bookings[numBookings].paymentStatus = "Paid"; // Assuming payment
is made immediately
                    numBookings++;
                }
                else
                {
                    cout << "\nMaximum booking limit reached. Cannot book more
tickets.\n";
                }
                addOn.orderAddOns();
                                        double total = addOn.getTotalPrice() +
busList[i].getTicketPrice();

                printTicket(bookings[numBookings - 1], total);
            }
            else
            {
                cout << "\nSeat already booked. Please choose another seat.\n";
            }
        }
        else
        {
            cout << "\nInvalid seat number. Please choose a valid seat.\n";
        }
        return;
    }
  }

  cout << "\nBus ID not found. Please enter a valid Bus ID.\n";
}

void Passenger::cancelBooking()
{
  string busID;
  int seatNumber;
```

```
    cout << "\n=============================================\n";
    cout << "||            CANCEL BOOKING              ||\n";
    cout << "=============================================\n\n";
    cout << "Enter Bus ID: ";
    cin >> busID;

    // Find the bus in the booking history
    for (int i = 0; i < numBookings; ++i)
    {
        if (bookings[i].busID == busID)
        {
            // Display seat number
            seatNumber = stoi(bookings[i].seatNumber);
            cout << "Seat Number: " << seatNumber << endl;

            // Cancel booking from bus and booking history
            for (int j = 0; j < numBuses; ++j)
            {
                if (busList[j].getBusID() == busID)
                {
                    busList[j].cancelSeat(seatNumber);
                    break;
                }
            }

            // Shift bookings in array to remove the selected booking
            for (int k = i; k < numBookings - 1; ++k)
            {
                bookings[k] = bookings[k + 1];
            }
            numBookings--;

            cout << "\nBooking canceled successfully!\n";
            return;
        }
    }

    cout << "\nBus ID not found in booking history. Please enter a valid Bus ID.\n";
}

void Passenger::viewBuses()
{
    cout << "\n=============================================\n";
    cout << "||              VIEW BUSES                 ||\n";
    cout << "=============================================\n\n";
```

```
    cout << setw(10) << "Bus ID" << setw(15) << "Operator" << setw(15) << "Depart Date"
<< setw(15) << "Depart Time"
         << setw(15) << "Arrival Time" << setw(20) << "Boarding Point" << setw(20) <<
"Drop-off Point" << setw(10) << "Seats" << setw(10) << "Price" << endl;

    for (int i = 0; i < numBuses; ++i)
    {
        busList[i].displayBusDetails();
    }
}

void Passenger::rateBooking()
{
    string busID;
    string rating;

    cout << "\n==============================================\n";
    cout << "||            RATE BOOKING                ||\n";
    cout << "==============================================\n\n";
    cout << "Enter Bus ID to rate: ";
    cin >> busID;

    // Find the bus in the booking history
    for (int i = 0; i < numBookings; ++i)
    {
        if (bookings[i].busID == busID)
        {
            // Display details
            cout << "Bus ID: " << busID << endl;
            cout << "Rating (out of 5): ";
            cin >> rating;

            // Update rating in booking history
            bookings[i].rating = rating;
            cout << "\nRating submitted successfully!\n";
            return;
        }
    }

    cout << "\nBus ID not found in booking history. Please enter a valid Bus ID.\n";
}

void Passenger::printTicket(const Booking &booking, double total)
{
    ofstream ticketFile("Ticket.txt");
```

```
    if (!ticketFile)
    {
        cerr << "Error creating file!" << endl;
        return;
    }

    ticketFile << "EasyBus Booking Confirmation\n\n";
    ticketFile << "Passenger Info:\n";
    ticketFile << "Name: " << username << endl;
    ticketFile << "Payment Status: " << booking.paymentStatus << endl;
    ticketFile << "\nBus Details:\n";
    ticketFile << "Bus ID: " << booking.busID << endl;
    ticketFile << "Seat Number: " << booking.seatNumber << endl;

    for (int i = 0; i < numBuses; ++i)
    {
        if (busList[i].getBusID() == booking.busID)
        {
            ticketFile << "Operator: " << busList[i].getBusOperator() << endl;
            ticketFile << "Departure Date: " << busList[i].getDepartureDate() << endl;
            ticketFile << "Departure Time: " << busList[i].getDepartureTime() << endl;
            ticketFile << "Arrival Time: " << busList[i].getArrivalTime() << endl;
            ticketFile << "Boarding Point: " << busList[i].getBoardingPoint() << endl;
            ticketFile << "Drop-off Point: " << busList[i].getDropOffPoint() << endl;
            ticketFile << "Ticket Price: RM" << busList[i].getTicketPrice() << endl;
            ticketFile << "Add Ons Price: RM" << total - busList[i].getTicketPrice() << endl;
            ticketFile << "\n\n\nTotal Price: RM" << total << endl;
            break;
        }
    }

    ticketFile.close();
    cout << "Ticket has been printed to 'Ticket.txt'\n";
}

void Passenger::displayPassengerDetails() const {
        cout << "Passenger Details:\\nName: " << name << endl;
        ticket->displayTicketDetails();
    }


void mainMenu();
void adminLogin(Admin admins[], int numAdmins);
```

```
void passengerSignUp(Passenger passengers[], int &numPassengers); // Pass
numPassengers by reference
void passengerLogin(Passenger passengers[], int numPassengers);

Admin admins[10];
Passenger passengers[10];

void mainMenu()
{
   int choice;
   cout << "======================================== Welcome to EasyBus!
========================================";
          cout << R"(


                                ___                 _ _    ___
                               | __|   __ _     ___    | || |  | _ )   _ _     ___
                               | _|   / _` |   (_-<     \_, |  | _ \   | +| |   (_-<
                               |___|  \__,_|   /__/_   _|__/   |___/   \_,_|   /__/_
                              _|""""""|_|""""""|_|""""""|_| """""|_|""""""|_|""""""|_|""""""|
                              "`-0-0-'"`-0-0-'"`-0-0-'"`-0-0-'"`-0-0-'"`-0-0-'"`-0-0-'
                    )" << '\n';
          cout <<
"================================================================================
========================\n\n";

   cout << "        You are using EasyBook as...?          \n\n";
   cout << "1. Admin\n";
   cout << "2. Passenger\n";
   cout << "3. Exit\n";
   cout << "\nOption: ";
   cin >> choice;

   switch (choice)
   {
   case 1:
      adminLogin(admins, 1);
      break;
   case 2:
      int passengerChoice;
      cout << "\n<<<<<<<<<<<<   Welcome, Passenger!  >>>>>>>>>>>>>\n\n";
      cout << "1. Sign Up\n";
      cout << "2. Login\n";
      cout << "\nOption: ";
      cin >> passengerChoice;
      if (passengerChoice == 1)
```

```
        {
            passengerSignUp(passengers, numPassengers); // Pass numPassengers
        }
        else if (passengerChoice == 2)
        {
            passengerLogin(passengers, numPassengers); // Pass numPassengers
        }
        else
        {
            cout << "Invalid option! Please input correct option.\n";
        }
        break;
    case 3:
        exit(0);
    default:
        cout << "Invalid option! Please input correct option.\n";
    }
}

void adminLogin(Admin admins[], int numAdmins)
{
    string username, password;
    cout << "\n==============================================\n";
    cout << "||              ADMIN LOGIN                 ||\n";
    cout << "==============================================\n\n";
    cout << "Admin Username: ";
    cin >> username;
    cout << "Admin Password: ";
    cin >> password;

    for (int i = 0; i < numAdmins; ++i)
    {
        if (admins[i].getUsername() == username && admins[i].getPassword() ==
password)
        {
            admins[i].menu();
            return;
        }
    }
    cout << "Incorrect username or password. Please try again.\n";
}

void passengerSignUp(Passenger passengers[], int &numPassengers)
{
    string username, password;
```

```
    cout << "\n=============================================\n";
    cout << "||          PASSENGER SIGNUP           ||\n";
    cout << "=============================================\n\n";
    cout << "Username: ";
    cin >> username;
    cout << "Password: ";
    cin >> password;

    // Check if the username is already taken
    for (int i = 0; i < numPassengers; ++i)
    {
        if (passengers[i].getUsername() == username)
        {
            cout << "\nUsername already exists. Please choose a different username.\n";
            return;
        }
    }

    // Add new passenger to the array
    if (numPassengers < MAX_PASSENGERS)
    {
        passengers[numPassengers++] = Passenger(username, password);
        cout << "\nAccount created successfully!\n";
    }
    else
    {
        cout << "\nCannot add more passengers. Maximum limit reached.\n";
    }
}

void passengerLogin(Passenger passengers[], int numPassengers)
{
    string username, password;
    cout << "\n=============================================\n";
    cout << "||          PASSENGER LOGIN            ||\n";
    cout << "=============================================\n\n";
    cout << "Username: ";
    cin >> username;
    cout << "Password: ";
    cin >> password;

    // Check credentials
    for (int i = 0; i < numPassengers; ++i)
    {
```

```
        if (passengers[i].getUsername() == username && passengers[i].getPassword() ==
password)
        {
            passengers[i].menu();
            return;
        }
    }
    cout << "Incorrect username or password. Please try again.\n";
}

int main()
{
    admins[0] = Admin("admin", "admin123");

    busList[numBuses++] = Bus("BUS034", "Sani", "2024-12-01", "08:00", "12:00",
"Larkin", "TBS", 40, 27);
    busList[numBuses++] = Bus("BUS123", "SPBumi", "2024-03-25", "10:40", "15:00",
"TBS", "TSK", 30, 33);
    busList[numBuses++] = Bus("BUS112", "Mayang", "2024-11-02", "14:50", "17:00",
"TSK", "TBS", 40, 30);

    while (true)
    {
        mainMenu();
    }

    return 0;
}
```